
Sistema de Gestão de Locações de Imóveis

RCP Data Imob — Documentação Completa

Disciplina: Laboratório de Programação 3

Professor: Matheus Vanzan

Integrantes:

Marcell Parra Araújo B. Silva

Rafael Vargas Carvalheira

Ruan Pablo Rodrigues

Data: Março de 2026

Sumário

1. Apresentação do Projeto
2. Objetivo do Sistema
3. Escopo Funcional
4. Funcionalidade Seleccionada para o Mobile
5. Arquitetura da Solução
6. Stack Tecnológica
7. Modelo de Dados
8. Estrutura do Backend
9. API REST
10. Banco de Dados e Docker (Entregável 3)
11. Frontend e Docker (Entregável 4)
12. Capturas de Tela do Frontend
13. Execução do Ambiente
14. Considerações sobre a Implementação
15. Conclusão
16. Referências Internas do Projeto

1. Apresentação do Projeto

O projeto **Sistema de Gestão de Locações de Imóveis** consiste em uma aplicação web desenvolvida para apoiar a administração de imóveis destinados à locação. A proposta central do sistema é reunir, em uma única plataforma, as operações de cadastro de imóveis e clientes, controle de reservas, verificação de disponibilidade e acompanhamento financeiro da operação.

A solução foi concebida em arquitetura cliente-servidor desacoplada, com backend em API REST e frontend em SPA (Single Page Application). Essa separação facilita a manutenção do sistema, melhora a organização do código e permite reutilização futura da API por aplicações mobile.

Além disso, o projeto foi estruturado para atender aos requisitos práticos da disciplina, contemplando desde a definição do tema e da stack tecnológica até a implementação de serviços executáveis em ambiente containerizado e a disponibilização de endpoints REST organizados por entidade de negócio.

2. Objetivo do Sistema

O objetivo do sistema é oferecer uma solução completa para o gerenciamento de locações de imóveis, permitindo:

- Cadastrar e manter o portfólio de imóveis disponíveis;
- Cadastrar clientes e seus dados de contato;
- Criar, consultar, editar e cancelar locações;
- Verificar disponibilidade de imóveis com base em período e restrições de data;
- Categorizar imóveis por características relevantes;
- Registrar receitas e despesas associadas à operação;
- Acompanhar indicadores financeiros e operacionais.

Com isso, o sistema busca tornar o processo de administração de locações mais organizado, rastreável e eficiente.

3. Escopo Funcional

O sistema foi planejado em torno de três módulos principais, detalhados a seguir.

3.1 Gestão de Imóveis com Categorias

Módulo responsável pelo cadastro e manutenção dos imóveis disponíveis para locação.

- Cadastro completo de imóveis com informações como título, descrição, endereço, cidade, estado, CEP, número de quartos, banheiros, vagas de garagem, área, valor de aluguel e valor de venda;
- Associação de múltiplas categorias por imóvel (Piscina, Ar-condicionado, Pet-friendly, etc.);

- Criação de novas categorias durante o fluxo de cadastro;
- Busca e filtros por título, cidade e status de disponibilidade;
- Visualização em cards com informações resumidas;
- Edição e exclusão com validações de integridade.

3.2 Sistema de Locações com Controle de Status

Módulo central da aplicação, responsável pelo controle das locações.

- Listagem de locações com imóvel, cliente, datas, valor mensal e status (ativa/inativa);
- Formulário de nova locação com selects dinâmicos de imóvel e cliente;
- Encerramento de locações ativas via PATCH;
- Indicação visual de locação ativa vs. inativa com badges semânticos;
- Cálculo e exibição de valor mensal em formato de moeda brasileira.

3.3 Controle Financeiro com Dashboard Analítico

Módulo que consolida receitas e despesas associadas à operação.

- Registro de lançamentos financeiros com tipo (receita/despesa), valor, data de vencimento e locação associada;
- Filtros por status (pendente/pago/atrasado);
- Cards de resumo com total de receitas e total de despesas;
- Registro de pagamento via PATCH;
- Dashboard com KPIs consolidados e gráfico de evolução financeira dos últimos 6 meses;
- Listagem das últimas locações registradas.

4. Funcionalidade Seleccionada para o Mobile

A funcionalidade escolhida para adaptação em ambiente mobile foi a **Busca de Disponibilidade e Criação de Reserva**. Essa escolha foi motivada por se tratar de um fluxo frequente para administradores de imóveis, especialmente em situações fora do escritório.

Recursos previstos para o mobile:

- Tela com campos de data de entrada e saída para busca rápida;
- Listagem de imóveis disponíveis no período informado;
- Exibição resumida com tipo, capacidade e valor estimado;
- Tela de detalhes do imóvel com categorias;
- Confirmação de reserva com seleção de cliente;
- Feedback imediato em caso de sucesso ou conflito de datas.

Justificativa: A consulta de disponibilidade e a confirmação de reservas são tarefas que exigem agilidade. Uma versão mobile desse fluxo reduz a dependência de computador e acelera o atendimento ao cliente.

5. Arquitetura da Solução

A aplicação adota uma arquitetura dividida em três camadas principais:

- **Banco de dados:** responsável pelo armazenamento persistente dos dados do sistema;
- **Backend:** responsável pela lógica de negócio, exposição dos endpoints e comunicação com o banco;
- **Frontend:** responsável pela interface visual e interação com o usuário.

Os serviços são orquestrados com Docker Compose, o que simplifica a execução do ambiente de desenvolvimento e padroniza a configuração entre diferentes máquinas.

5.1 Serviços e Portas

Serviço	Imagem / Tecnologia	Porta	Descrição
db	postgres:13-alpine	5432	Banco de dados PostgreSQL
backend	node:18-alpine (build)	3000	API REST Node.js / Express
frontend	node:20-alpine (build)	5173	Interface React / Vite

6. Stack Tecnológica

6.1 Backend

Tecnologia	Versão	Função no projeto
Node.js	v18+	Runtime JavaScript no servidor
Express.js	4.18.2	Framework para API REST
PostgreSQL	v13+	Banco de dados relacional principal
pg	8.11.3	Driver PostgreSQL para Node.js
cors	2.8.5	Middleware para Cross-Origin Requests

6.2 Frontend

Tecnologia	Versão	Função no projeto
React	v18.2	Construção da interface SPA
Vite	v6.0	Build tool e servidor de desenvolvimento
React Router DOM	v7.0	Roteamento client-side
Tailwind CSS	v3.4	Estilização responsiva com utility-first
Recharts	v2.12	Gráficos do dashboard financeiro
Lucide React	v0.400	Biblioteca de ícones
Axios	v1.7	Comunicação HTTP com a API

6.3 Infraestrutura

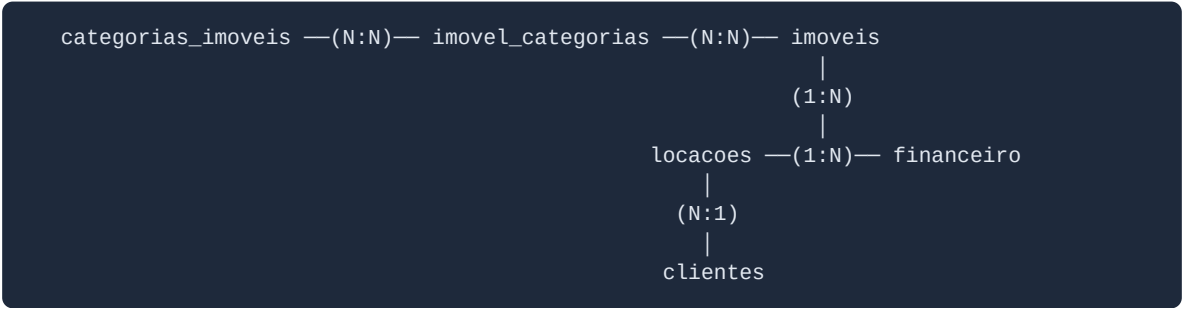
Tecnologia	Função
Docker	Containerização dos serviços
Docker Compose	Orquestração multi-container
Google Fonts	Tipografia (Plus Jakarta Sans + DM Sans)

7. Modelo de Dados

As tabelas principais do sistema estão organizadas conforme a estrutura abaixo.

Tabela	Conteúdo
usuarios	Dados de acesso e perfil dos administradores
imoveis	Cadastro dos imóveis com atributos, valores e status
categorias_imoveis	Categorias personalizadas para classificação
imovel_categorias	Relacionamento N:N entre imóveis e categorias
clientes	Cadastro de locatários
locacoes	Contratos de locação, datas, valores e status
financeiro	Lançamentos de receitas e despesas

7.1 Diagrama de Relacionamentos



7.2 Detalhamento das Tabelas

Tabela: imoveis

Coluna	Tipo	Restrições	Descrição
id	SERIAL	PRIMARY KEY	Identificador único
titulo	VARCHAR(255)	NOT NULL	Título do imóvel
descricao	TEXT	—	Descrição detalhada
endereco	VARCHAR(500)	NOT NULL	Endereço completo
cidade	VARCHAR(100)	—	Cidade
estado	VARCHAR(2)	—	UF
cep	VARCHAR(10)	—	CEP
valor_aluguel	NUMERIC(12,2)	—	Valor mensal de aluguel
valor_venda	NUMERIC(12,2)	—	Valor de venda
area	NUMERIC(10,2)	—	Área em m²
quartos	INTEGER	DEFAULT 0	Número de quartos
banheiros	INTEGER	DEFAULT 0	Número de banheiros
vagas_garagem	INTEGER	DEFAULT 0	Vagas de garagem
disponivel	BOOLEAN	DEFAULT TRUE	Disponível para locação
criado_em	TIMESTAMP	DEFAULT NOW()	Data de cadastro

Tabela: clientes

Coluna	Tipo	Restrições	Descrição
id	SERIAL	PRIMARY KEY	Identificador único
nome	VARCHAR(255)	NOT NULL	Nome completo

Coluna	Tipo	Restrições	Descrição
cpf	VARCHAR(14)	UNIQUE	CPF (000.000.000-00)
email	VARCHAR(255)	—	E-mail de contato
telefone	VARCHAR(20)	—	Telefone
endereco	VARCHAR(500)	—	Endereço do cliente
criado_em	TIMESTAMP	DEFAULT NOW()	Data de cadastro

Tabela: locacoes

Coluna	Tipo	Restrições	Descrição
id	SERIAL	PRIMARY KEY	Identificador único
imovel_id	INTEGER	FK → imoveis(id)	Imóvel locado
cliente_id	INTEGER	FK → clientes(id)	Locatário
data_inicio	DATE	NOT NULL	Início do contrato
data_fim	DATE	—	Fim do contrato
valor_mensal	NUMERIC(12,2)	NOT NULL	Valor mensal
ativa	BOOLEAN	DEFAULT TRUE	Locação em vigor
criado_em	TIMESTAMP	DEFAULT NOW()	Data de registro

Tabela: financeiro

Coluna	Tipo	Restrições	Descrição
id	SERIAL	PRIMARY KEY	Identificador único
locacao_id	INTEGER	FK → locacoes(id)	Locação relacionada
tipo	VARCHAR(50)	NOT NULL	receita / despesa
valor	NUMERIC(12,2)	NOT NULL	Valor do lançamento
data_vencimen to	DATE	NOT NULL	Data de vencimento
data_pagament o	DATE	—	Data de pagamento
status	VARCHAR(50)	DEFAULT 'pendente'	pendente / pago / atrasado
criado_em	TIMESTAMP	DEFAULT NOW()	Data de registro

8. Estrutura do Backend

O backend foi organizado de forma modular, com separação entre ponto de entrada, acesso ao banco e rotas por entidade.

8.1 Organização do Projeto

Caminho	Descrição
backend/src/index.js	Ponto de entrada; inicializa o Express e registra as rotas
backend/src/db/pool.js	Pool de conexão com PostgreSQL
backend/src/routes/imoveis.js	Rotas CRUD de imóveis
backend/src/routes/clientes.js	Rotas CRUD de clientes
backend/src/routes/locacoes.js	Rotas CRUD de locações
backend/src/routes/categorias.js	Rotas CRUD de categorias
backend/src/routes/financeiro.js	Rotas CRUD de lançamentos financeiros
backend/src/routes/status.js	Endpoint de healthcheck

8.2 Conexão com o Banco de Dados

A comunicação com o PostgreSQL é realizada por meio de um pool de conexões (pg.Pool). Esse mecanismo evita a abertura e o fechamento de uma nova conexão a cada requisição, reduzindo overhead e melhorando a eficiência do backend.

```
const pool = new Pool({
  host: process.env.DB_HOST || 'localhost',
  port: process.env.DB_PORT || 5432,
  database: process.env.DB_NAME || 'rcp_data_imob',
  user: process.env.DB_USER || 'admin',
  password: process.env.DB_PASSWORD || 'admin123',
});
```

8.3 Variáveis de Ambiente

Variável	Valor padrão	Descrição
DB_HOST	db	Host do serviço PostgreSQL no Compose
DB_PORT	5432	Porta do PostgreSQL
DB_NAME	rcp_data_imob	Nome do banco de dados
DB_USER	admin	Usuário de acesso

Variável	Valor padrão	Descrição
DB_PASSWORD	admin123	Senha de acesso

9. API REST

A API foi construída com Node.js e Express, seguindo princípios REST para organização de recursos, uso semântico dos verbos HTTP e respostas padronizadas em JSON. O servidor escuta na porta 3000, e as rotas estão separadas por entidade.

9.1 Endpoint de Healthcheck

Método	Rota	Descrição	Resposta
GET	/status	Verifica status da aplicação e do banco	{ "status": "ok", "banco": "conectado" }

9.2 Endpoints de Imóveis

Método	Rota	Descrição
GET	/imoveis	Lista todos os imóveis cadastrados
GET	/imoveis/:id	Retorna um imóvel específico pelo ID
POST	/imoveis	Cria um novo imóvel
PUT	/imoveis/:id	Atualiza os dados de um imóvel
DELETE	/imoveis/:id	Remove um imóvel do sistema

9.3 Endpoints de Clientes

Método	Rota	Descrição
GET	/clientes	Lista todos os clientes cadastrados
GET	/clientes/:id	Retorna um cliente específico pelo ID
POST	/clientes	Cadastra um novo cliente
PUT	/clientes/:id	Atualiza os dados de um cliente
DELETE	/clientes/:id	Remove um cliente do sistema

9.4 Endpoints de Locações

Método	Rota	Descrição
GET	/locacoes	Lista todas as locações registradas
GET	/locacoes/:id	Retorna uma locação específica
POST	/locacoes	Cria um novo contrato de locação
PATCH	/locacoes/:id/encerrar	Encerra uma locação ativa

9.5 Endpoints de Categorias

Método	Rota	Descrição
GET	/categorias	Lista todas as categorias disponíveis
POST	/categorias	Cria uma nova categoria
DELETE	/categorias/:id	Remove uma categoria do sistema

9.6 Endpoints Financeiros

Método	Rota	Descrição
GET	/financeiro	Lista todos os lançamentos financeiros
POST	/financeiro	Registra um novo lançamento
PATCH	/financeiro/:id/pagar	Registra pagamento de um lançamento

9.7 Resumo Geral dos Endpoints

Ao todo, a API contempla **16 endpoints** organizados por entidade de negócio:

Entidade	GET	POST	PUT	PATCH	DELETE	Total
/status	1	—	—	—	—	1
/imoveis	2	1	1	—	1	5
/clientes	2	1	1	—	1	5
/locacoes	2	1	—	1	—	4
/categorias	1	1	—	—	1	3
/financeiro	1	1	—	1	—	3

10. Banco de Dados e Docker (Entregável 3)

O Entregável 3 consolidou a configuração do banco de dados PostgreSQL via Docker, a modelagem das entidades do sistema e a integração completa entre o backend Node.js/Express e o banco de dados. O ambiente é orquestrado pelo Docker Compose, garantindo portabilidade e facilidade de execução em qualquer máquina.

10.1 Status dos Requisitos

Requisito	Status
Banco de dados configurado no Docker (PostgreSQL 13)	Concluído
Modelagem das entidades e relacionamentos	Concluído
Backend integrado ao banco com persistência via API	Concluído
CRUD completo testado via Postman	Concluído
Documentação e capturas de tela	Concluído

10.2 Configuração do docker-compose.yml

```

services:
  db:
    image: postgres:13-alpine
    environment:
      POSTGRES_DB: rcp_data_imob
      POSTGRES_USER: admin
      POSTGRES_PASSWORD: admin123
    ports:
      - "5432:5432"
    volumes:
      - pgdata:/var/lib/postgresql/data
      - ./database/init.sql:/docker-entrypoint-initdb.d/init.sql
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U admin -d rcp_data_imob"]
      interval: 5s
      timeout: 5s
      retries: 5

  backend:
    build: ./backend
    ports:
      - "3000:3000"
    environment:
      DB_HOST: db
      DB_PORT: 5432
      DB_NAME: rcp_data_imob
      DB_USER: admin
      DB_PASSWORD: admin123
    depends_on:
      db:
        condition: service_healthy

  frontend:
    build: ./frontend
    ports:
      - "5173:5173"
    depends_on:
      - backend
    environment:
      - VITE_API_URL=http://localhost:3000

volumes:
  pgdata:

```

10.3 Inicialização Automática do Schema

O arquivo `database/init.sql` é montado no diretório de inicialização do PostgreSQL (`/docker-entrypoint-initdb.d/`), garantindo que as tabelas sejam criadas automaticamente na primeira execução do container.

10.4 Testes CRUD via Postman

Foram executados testes completos das 4 operações fundamentais (Create, Read, Update, Delete) via Postman contra a API. A coleção está disponível em

postman/RCP_Data_Imob.postman_collection.json.

Operação	Método	Endpoint	Status
Create	POST	/clientes	201 Created
Read (listar)	GET	/clientes	200 OK
Update	PUT	/clientes/:id	200 OK
Delete	DELETE	/clientes/:id	200 OK

O endpoint GET /status executa um SELECT NOW() contra o banco, confirmando que a conexão está ativa. Retorna HTTP 200 com banco conectado ou HTTP 503 em caso de falha.

```
{
  "status": "ok",
  "servico": "RCP Data Imob API",
  "versao": "1.0.0",
  "banco": "conectado",
  "timestamp": "2026-03-17T01:00:00.000Z"
}
```

11. Frontend e Docker (Entregável 4)

O Entregável 4 implementou o frontend completo do sistema com React, configurado e executável via Docker, consumindo dados reais da API REST do backend.

11.1 Requisitos Atendidos

Requisito	Status
Frontend configurado e rodando no Docker	Concluído
Tela inicial do sistema criada (Dashboard)	Concluído
Navegação mínima entre telas (menu/rotas)	Concluído
Frontend consumindo dados da API (múltiplas chamadas)	Concluído
Exibição de dados vindos da API (listas, KPIs, gráficos)	Concluído
Documentação correspondente e capturas de tela	Concluído

11.2 Configuração Docker do Frontend

Dockerfile (frontend/Dockerfile):

```
FROM node:20-alpine
WORKDIR /app
COPY package.json ./
RUN npm install
COPY . .
EXPOSE 5173
CMD ["npm", "vite", "--host", "0.0.0.0"]
```

Proxy do Vite (vite.config.js):

O Vite foi configurado com proxy reverso para redirecionar chamadas /api para o backend no Docker, evitando problemas de CORS em ambiente containerizado:

```
server: {
  host: '0.0.0.0',
  port: 5173,
  proxy: {
    '/api': {
      target: 'http://backend:3000',
      changeOrigin: true,
      rewrite: (path) => path.replace(/^\/api/, '')
    }
  }
}
```

11.3 Estrutura do Frontend

```
frontend/
├─ index.html           # Entrada HTML com Google Fonts
├─ package.json         # Dependências React + Vite + Tailwind
├─ vite.config.js       # Configuração Vite com proxy
├─ tailwind.config.js   # Tema customizado (cores, fontes, sombras)
├─ postcss.config.js    # PostCSS com Tailwind e Autoprefixer
├─ Dockerfile           # Imagem Node 20 Alpine
├─ .dockerignore        # Ignora node_modules e dist
├─ src/
│  ├─ main.jsx          # Ponto de entrada React
│  ├─ App.jsx           # Roteamento com React Router DOM
│  ├─ index.css         # Estilos globais, animações, Tailwind
│  ├─ api/
│  │  └─ axios.js       # Instância Axios centralizada
│  ├─ components/
│  │  ├─ Layout.jsx     # Sidebar fixa + área de conteúdo
│  │  ├─ Modal.jsx      # Modal genérico com animação
│  │  ├─ StatusBadge.jsx # Badge semântico de status
│  │  ├─ Skeleton.jsx   # Loading skeletons
│  │  └─ EmptyState.jsx  # Estado vazio com ícone
│  └─ pages/
│     ├─ Dashboard.jsx  # KPIs, gráfico, últimas locações
│     ├─ Imoveis.jsx    # Cards com filtros, CRUD
│     ├─ Clientes.jsx   # Tabela com busca, CRUD
│     ├─ Locacoes.jsx  # Tabela, selects dinâmicos
│     └─ Financeiro.jsx # Cards resumo, filtro, pagamento
```

11.4 Decisões de Design

O frontend foi desenvolvido com identidade visual própria de sistema imobiliário profissional.

Tipografia

- **Títulos:** Plus Jakarta Sans (weight 700-800) — display bold com personalidade
- **Corpo:** DM Sans — legibilidade moderna para textos e labels
- Carregamento via Google Fonts no index.html

Paleta de Cores

- **Sidebar:** Dark navy (#0F172A) com navegação em tons de slate
- **Cor primária (brand):** Teal (#0D9488 / #115E59) para CTAs e destaques
- **Fundo do conteúdo:** Off-white (#F8FAFC) com pattern sutil de dots
- **Cards:** Branco com sombras em camadas e hover com elevação

Layout

- Sidebar fixa à esquerda (260px) com navegação vertical e ícones Lucide
- Indicador lateral teal na rota ativa
- Conteúdo principal com max-width 1280px e padding generoso
- Cards com border-radius 14px e espaçamento interno de 24px

Micro-interações e Motion

- Stagger animation nos cards ao carregar (fadeInUp com delay incremental)
- Hover nos cards com transform scale + shadow-lg
- Modais com fade-in + scale sutil + backdrop blur
- Loading states com skeleton screens (shimmer animation)
- Transições suaves em hover da sidebar

Componentes Visuais

- Status badges com cores semânticas (ativo=verde, inativo=cinza, pendente=amarelo, pago=verde, atrasado=vermelho)
- Botões com hierarquia clara: primário (filled teal), secundário (outlined), ghost
- Inputs com labels e espaçamento adequado
- Empty states com ícone Lucide + texto convidativo
- Tabelas com hover highlight e linhas alternadas

11.5 Telas Implementadas

Rota	Tela	Funcionalidades
/	Dashboard	4 KPIs, gráfico de barras (Recharts) com 6 meses, últimas locações
/imoveis	Imóveis	Cards com filtros (cidade/status/busca), CRUD completo via modal
/clientes	Clientes	Tabela com busca por nome, CRUD completo via modal
/locacoes	Locações	Tabela com selects de imóvel/cliente, encerramento de locação
/financeiro	Financeiro	Cards receitas/despesas, filtro por status, ação de pagamento

11.6 Consumo da API

O frontend consome dados reais da API REST do backend em todas as 5 telas:

Tela	Endpoints consumidos
Dashboard	GET /imoveis, GET /clientes, GET /locacoes, GET /financeiro
Imóveis	GET /imoveis, POST /imoveis, PUT /imoveis/:id, DELETE /imoveis/:id
Clientes	GET /clientes, POST /clientes, PUT /clientes/:id, DELETE /clientes/:id
Locações	GET /locacoes, GET /imoveis, GET /clientes, POST /locacoes, PATCH /locacoes/:id/encerrar
Financeiro	GET /financeiro, GET /locacoes, POST /financeiro, PATCH /financeiro/:id/pagar

Todas as chamadas são feitas via instância centralizada do Axios (src/api/axios.js) com baseURL: '/api' e proxy configurado no Vite para o backend.

12. Capturas de Tela do Frontend

A seguir são apresentadas as capturas de tela das 5 telas principais do sistema, documentando o estado visual de cada módulo implementado.

12.1 Dashboard

Tela inicial com visão geral do sistema: 4 cards de KPIs (Total de Imóveis, Total de Clientes, Locações Ativas, Receita do Mês), gráfico de evolução financeira dos últimos 6 meses (Recharts) e lista das últimas locações.

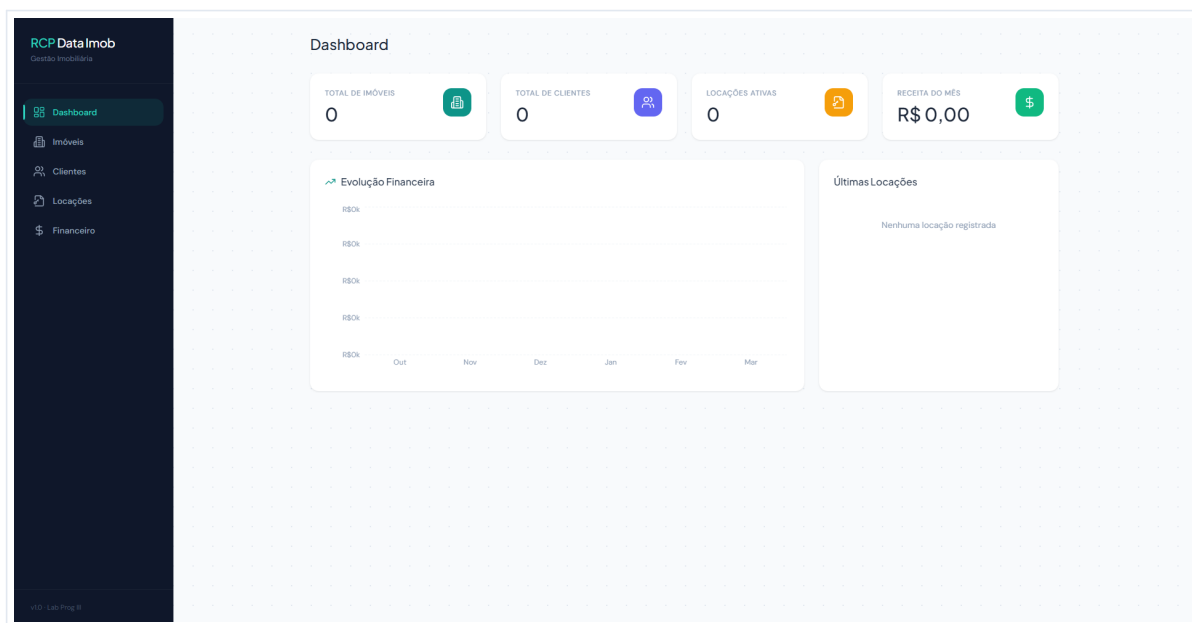


Figura 1 — Dashboard com KPIs, gráfico de evolução financeira e últimas locações

12.2 Imóveis

Tela de gestão de imóveis com barra de busca por título, filtros por cidade e status de disponibilidade, e botão "Novo Imóvel" para cadastro. Os imóveis são exibidos em cards com informações resumidas.

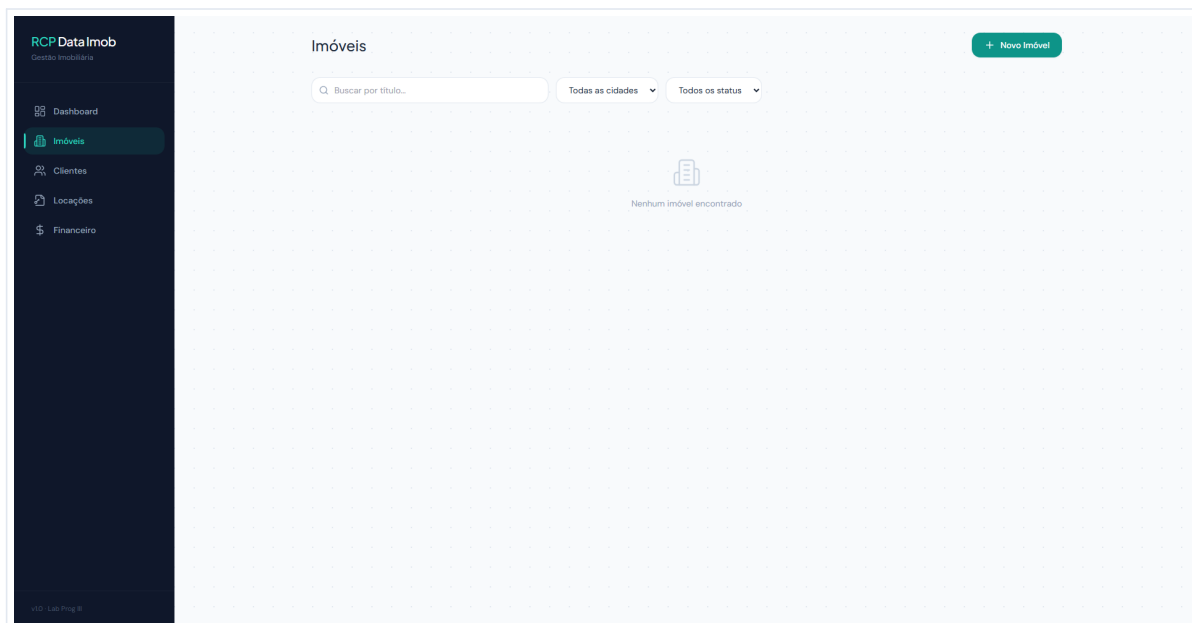


Figura 2 — Listagem de imóveis com filtros e busca

12.3 Clientes

Tela de gestão de clientes com busca por nome e botão "Novo Cliente". Os dados são apresentados em tabela com colunas Nome, CPF, Email, Telefone e ações de edição/exclusão.

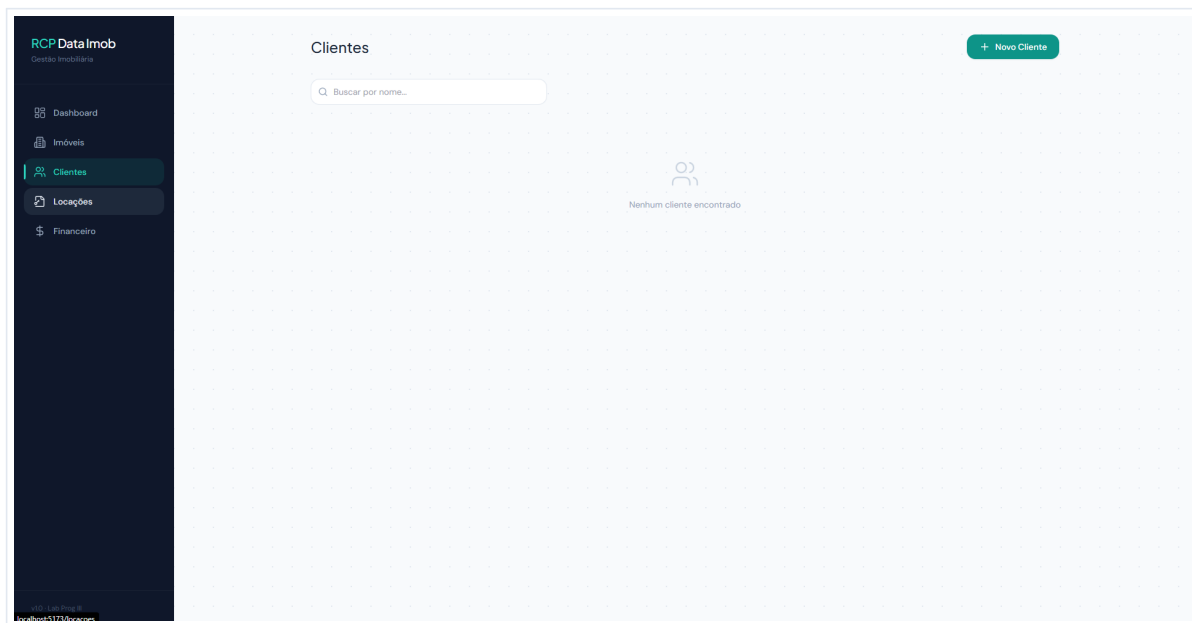


Figura 3 — Listagem de clientes com busca por nome

12.4 Locações

Tela de gestão de locações com listagem de contratos contendo imóvel, cliente, datas, valor mensal e status (ativo/inativo). Botão "Nova Locação" para registro com selects dinâmicos.

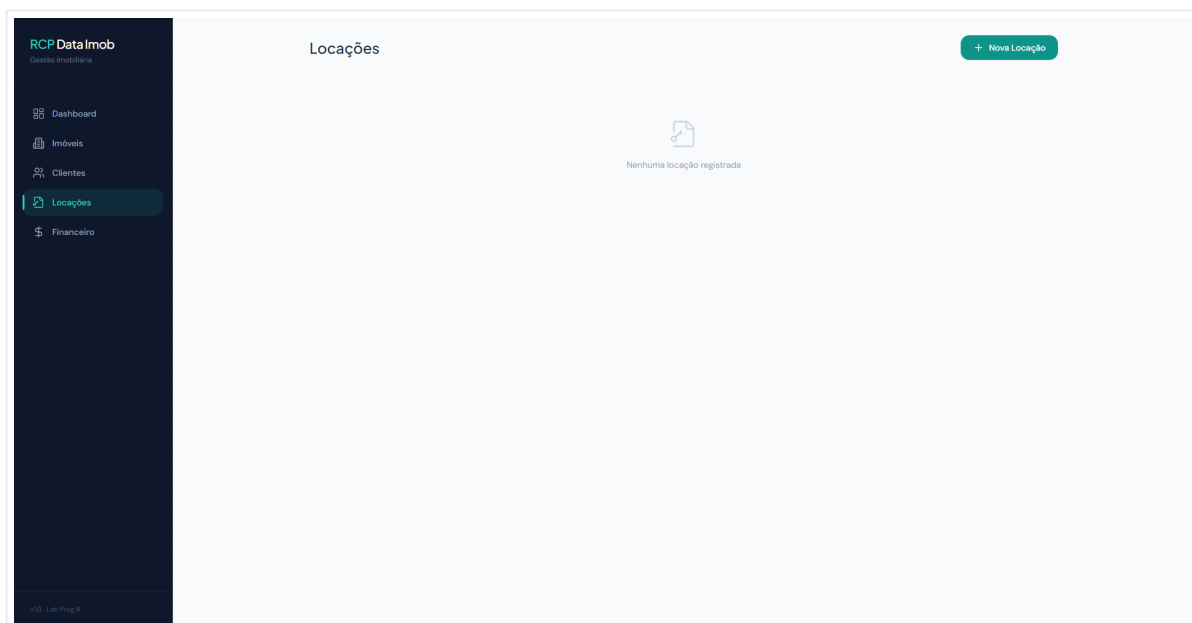


Figura 4 — Listagem de locações com status e ação de encerramento

12.5 Financeiro

Tela financeira com cards de resumo (Total Receitas e Total Despesas), filtro por status (pendente/pago/atrasado), e tabela de lançamentos com ação de pagamento.

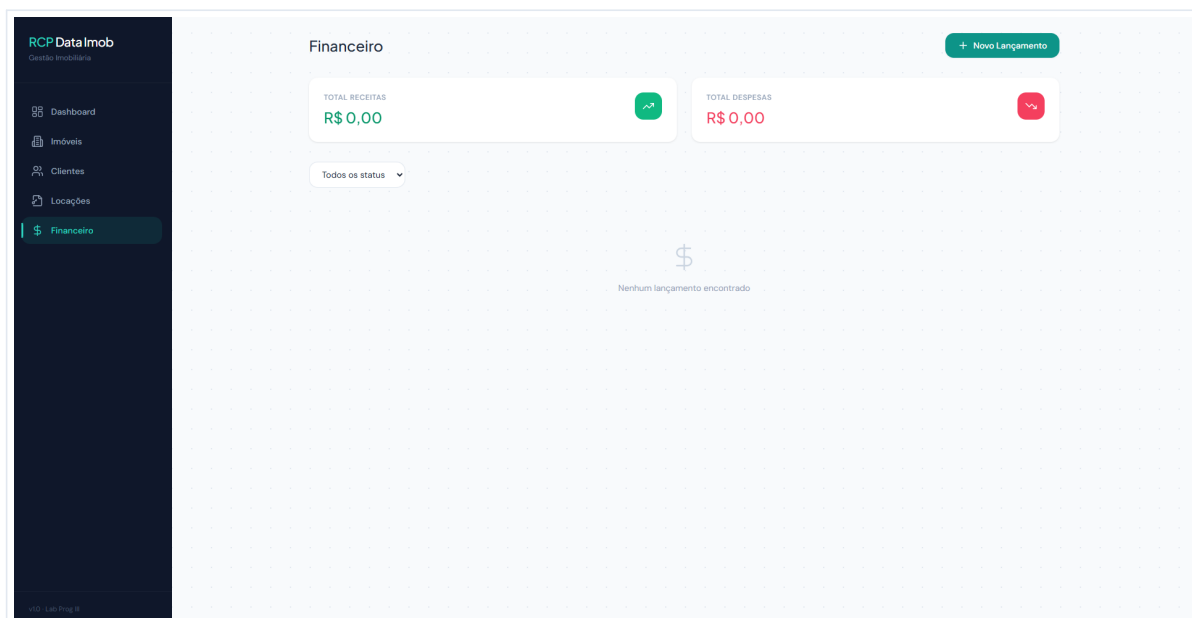


Figura 5 — Painel financeiro com resumo e filtros por status

13. Execução do Ambiente

O projeto foi preparado para execução com Docker Compose, permitindo subir todos os serviços de forma integrada.

13.1 Como Subir o Ambiente

```
# Subir todos os serviços (build + start em background)
docker compose up --build -d

# Verificar status dos containers
docker compose ps

# Ver logs de um serviço específico
docker compose logs frontend
docker compose logs backend
```

13.2 Como Derrubar o Ambiente

```
# Parar os serviços
docker compose down

# Remover também os volumes (apaga dados do banco)
docker compose down -v
```

13.3 Acessos

Serviço	URL
Frontend	http://localhost:5173
API REST	http://localhost:3000
PostgreSQL	localhost:5432

14. Considerações sobre a Implementação

A proposta do sistema demonstra uma evolução consistente ao longo dos entregáveis:

- No **Entregável 0**, foi definida a proposta do sistema, seu escopo funcional, a stack e a visão geral das telas;
- No **Entregável 2**, foi consolidada a implementação da API REST, com organização modular, integração com PostgreSQL e endpoints CRUD por entidade;
- No **Entregável 3**, foi configurado o banco de dados PostgreSQL via Docker, com modelagem completa das entidades, inicialização automática do schema e validação de CRUD via Postman;

- No **Entregável 4**, foi implementado o frontend completo com React, Vite e Tailwind CSS, configurado no Docker com proxy para o backend, consumindo dados reais da API em todas as 5 telas do sistema.

Essa progressão evidencia um desenvolvimento estruturado, com preocupação tanto com a definição arquitetural quanto com a implementação prática.

15. Conclusão

O Sistema de Gestão de Locações de Imóveis (RCP Data Imob) foi projetado e implementado como uma solução web completa para atender às necessidades de uma administradora de imóveis. O sistema integra cadastro, locações, categorização e controle financeiro em uma única aplicação com interface visual profissional.

Do ponto de vista técnico, a solução utiliza tecnologias atuais e amplamente adotadas no desenvolvimento web, com separação clara entre frontend, backend e banco de dados. A utilização de Docker Compose contribui para a reprodutibilidade do ambiente, enquanto a API REST organizada por entidade favorece escalabilidade e manutenção.

O frontend implementado oferece uma experiência de usuário completa, com Dashboard analítico, CRUD funcional em todas as entidades, design responsivo com Tailwind CSS, gráficos interativos com Recharts, e micro-interações que conferem profissionalismo à interface.

Como trabalho acadêmico, o projeto demonstra domínio de conceitos importantes da disciplina, como modelagem de sistema, organização em camadas, definição de endpoints, integração com banco de dados, containerização com Docker e construção de interfaces modernas com React.

16. Referências Internas do Projeto

- **Entregável 0** — Definição do tema, escopo, stack e telas;
- **Entregável 2** — Implementação da API REST e organização do backend;
- **Entregável 3** — Banco de dados PostgreSQL + Docker + testes CRUD via Postman;
- **Entregável 4** — Frontend React + Vite + Tailwind CSS no Docker, consumindo API;
- **Coleção Postman** — `postman/RCP_Data_Imob.postman_collection.json`

Estrutura Completa do Projeto

```

.
├── docker-compose.yml          # Orquestração dos 3 serviços
├── database/
│   └── init.sql                # DDL - criação das tabelas
├── backend/
│   ├── Dockerfile
│   ├── package.json
│   └── src/
│       ├── index.js           # Entrada Express
│       ├── db/pool.js         # Pool de conexão PostgreSQL
│       └── routes/
│           ├── status.js      # GET /status
│           ├── imoveis.js     # CRUD imóveis
│           ├── clientes.js    # CRUD clientes
│           ├── locacoes.js    # Locações + PATCH encerrar
│           ├── categorias.js  # CRUD categorias
│           └── financeiro.js  # Financeiro + PATCH pagar
├── frontend/
│   ├── Dockerfile
│   ├── .dockerignore
│   ├── package.json
│   ├── vite.config.js
│   ├── tailwind.config.js
│   ├── postcss.config.js
│   ├── index.html
│   └── src/
│       ├── main.jsx
│       ├── App.jsx
│       ├── index.css
│       ├── api/axios.js
│       ├── components/
│       │   ├── Layout.jsx
│       │   ├── Modal.jsx
│       │   ├── StatusBadge.jsx
│       │   ├── Skeleton.jsx
│       │   └── EmptyState.jsx
│       └── pages/
│           ├── Dashboard.jsx
│           ├── Imoveis.jsx
│           ├── Clientes.jsx
│           ├── Locacoes.jsx
│           └── Financeiro.jsx
├── postman/
│   └── RCP_Data_Imob.postman_collection.json
└── docs/
    ├── Documentacao_Completa_RCP_Data_Imob.md
    └── screenshots/

```